

# HPC Assignment: Final Deadline

By Ahmad El Hage (s4851692)

## 1. Introduction

The report provides a description of MPI+CUDA parallel implementation of the co-clustering algorithm applied for spatio-temporal analysis of phenological data of the spring season. This parallel solution is based on MPI-only version described in Deadline 1 and uses CUDA to parallelize the computations on GPU of each computer. As before, MPI distributes the matrix columns among nodes, thus combining inter-node parallelism (MPI) with intra-node parallelism (CUDA) to reach peak efficiency. Also, bonus optimizations were implemented on top of the Deadline 2 MPI+CUDA baseline. Four additional features were implemented, targeting both GPU-side computation efficiency and CPU-side intra-node parallelism. All implementations maintain full and are benchmarked against the baseline to quantify their effect.

The proposed solution works by assigning row labels (time periods) and column labels (locations) to a 2-dimensional input matrix having dimensions  $40 \times 10,110,932$  (XL dataset). Every iteration of the algorithm includes the following operations: computation of cluster averages, row assignment, and column assignment. All these steps have been implemented using CUDA kernels to utilize GPU capabilities.

To validate correctness of the solution, the comparison script was used on all datasets (S, M, L, XL, XXL) and two configurations ( $3 \times 20$  and  $5 \times 100$ ). The parallel solutions generated the same result with respect to the NMI metric (1) as the serial ones.

## 2. MPI Parallelization

The MPI parallelization strategy distributes columns across nodes using MPI\_Scatterv. Each node owns a contiguous block of columns and performs all GPU computation on that block independently where possible. Row labels (40 entries) are broadcast to all nodes via MPI\_Bcast since they are tiny and must be identical everywhere.

Three MPI collective operations are performed per iteration: two MPI\_Allreduce calls to combine partial cluster sums and partial row-label distances, and one MPI\_Allreduce for convergence detection. The communication payloads are small (at most 500 doubles for  $5 \times 100$  labels), making communication overhead negligible relative to computation.

## 3. CUDA Parallelisation

Each MPI process offloads its local column computations to a single GPU. The matrix data and labels are uploaded to the GPU once before the main loop. Four CUDA kernels implement the three co-clustering steps.

### 3.1 Kernel 1 calculate\_cluster\_average (kernel\_cluster\_sum)

Each thread handles one matrix entry (row, col). The thread reads the entry value, identifies its cluster as  $\text{row\_labels}[\text{row}] * \text{num\_col\_labels} + \text{col\_labels}[\text{col}]$ , and uses atomicAdd to accumulate into partial sum and count arrays of size  $\text{num\_row\_labels} \times \text{num\_col\_labels}$ . Thread blocks of 256 threads are launched with enough blocks to cover all  $\text{num\_rows} \times \text{local\_cols}$  entries. After the kernel operation, partial sums are moved to the host and aggregated by using MPI\_Allreduce. After this, global sums are moved back to the GPU and kernel\_divide\_avg kernel divides each sum by count of elements in a cluster. It is done by one thread per cluster.

### 3.2 Kernel 2 update\_row\_labels (kernel\_row\_distances)

In this kernel each thread processes one (row, candidate\_label) combination. There are  $\text{num\_rows} * \text{num\_row\_labels}$  such combinations (up to  $40 * 5 = 200$  combinations). In this kernel each thread computes partial squared distance between values of the row and corresponding cluster averages for particular candidate\_label. Partial distances are moved to the host and summed by MPI\_Allreduce to get global

distances. By using only 40 operations the CPU chooses best label for each row and sets it in `row_labels`. As `row_labels` are modified the same way on all MPI ranks, no additional broadcasting is required.

### 3.3 Kernel 3 `update_col_labels` (`kernel_col_labels`)

Each thread handles one local column. For each candidate column label, the thread loops over all rows and accumulates the squared distance. It then assigns the label with the smallest distance using `atomicAdd` to count changed labels. This kernel requires no MPI communication: the cluster averages and row labels are already globally consistent on every GPU. The kernel launches one thread per local column with blocks of 256 threads.

### 3.4 Thread Block Configuration

All kernels employ a constant thread block size of 256 threads, which is a typical and efficient value for current NVIDIA GPUs. Block counts are determined by  $\lceil \text{work\_items} / 256 \rceil$ . For example, the `col_labels` kernel for the XL dataset, with 1 node, will have  $\lceil 10,110,932 / 256 \rceil \approx 39,496$  blocks, a feasible number for the GPU.

### 3.5 Memory Management

All device allocations are performed once before the main loop and freed afterwards. The matrix (float) and labels (int) are uploaded once. Per-iteration transfers are limited to: partial sums/counts ( $\leq 500$  doubles + 500 ints) copied out for Allreduce, global results copied back, and the updated `row_labels` (40 ints) copied back. These transfers are small and do not bottleneck performance.

### 3.6 Effect of Multiple Tasks per Node

An experiment was carried out with 32 MPI tasks per node and 12 nodes with the XL 5×100 dataset. It produced no performance gain; the runtime rose from 26.58s (1 task/node) to 31.54s (32 tasks/node). This proves that one MPI task alone can fully utilize the GPU. Multiple tasks vie for the same GPU resources, creating additional overhead but not increasing computational power. The ideal setting is thus 1 MPI task per node in the CUDA version.

## 4. Bonus Implementations

### 4.1 Previous Code Modifications

Following feedback on the initial CUDA implementation, a couple of changes were made to the source code.

#### 4.1.1 Row Label Selection Moved to GPU

The original implementation offloaded row label selection to the CPU: after the `MPI_Allreduce` on partial distances, each rank iterated over `h_global_dist` in a serial loop to find the minimum-distance cluster for each row, then copied the updated labels back to the device. This introduced an unnecessary CPU bottleneck and a redundant host-device transfer on every iteration. Therefore, a dedicated kernel, `kernel_select_row_labels`, was introduced to perform this step entirely on the GPU. Each thread handles one row, scanning `d_global_dist` to find the best label, writing the result directly into `d_row_labels`, and atomically accumulating both the changed-label count and the total distance. The only data returned to the host is two scalars one int and one double rather than a full `num_rows`-length label array per iteration.

#### 4.1.2 Refactored into Three Helper Functions

The `cluster_cuda` function previously contained the full iteration loop inline, exceeding 150 lines and mixing kernel launches, memory transfers, and MPI calls into a single monolithic body. To improve readability and match the structure of the reference serial implementation, the loop body was split into three self-contained functions:

*calculate\_cluster\_average* zeroes scratch buffers, launches the cluster-sum kernel, performs the D→H transfer, runs MPI\_Allreduce on sums and counts, transfers the global result back, and launches *kernel\_divide\_avg*.

*update\_row\_labels* launches *kernel\_row\_distances*, transfers partial distances to the host, runs MPI\_Allreduce, transfers global distances back to the device, and launches *kernel\_select\_row\_labels*. Returns the number of changed labels and the total distance as a pair.

*update\_col\_labels* launches *kernel\_col\_labels*, retrieves the local changed-count, and reduces it across ranks with MPI\_Allreduce. Returns the global changed-column count.

All device pointers and streams are bundled in a GpuContext struct passed by reference, so the functions share state without requiring long argument lists. The main loop in *cluster\_cuda* is reduced to three function calls plus the convergence check.

4.2 Three Kernel Variants

Three implementations of *kernel\_cluster\_sum* were developed, each using a different reduction strategy:

| Variant | Strategy           | Description                                                                                                                                                                                                                |
|---------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A       | Global Atomics     | Each thread atomicAdd directly to global memory. Simple but causes heavy contention when many threads target the same cluster slot.                                                                                        |
| B       | Shared Memory      | Each block accumulates into block-local shared memory first (fast, no DRAM latency), then flushes once per cluster per block to global. Reduces global atomic traffic by up to blockDim.x times.                           |
| C       | Warp Shuffle (ref) | Uses __shfl_down_sync to exchange values between warp lanes in registers, avoiding memory entirely. Benchmarked for reference. Not selected — non-uniform cluster distribution across warp lanes makes results unreliable. |

Figure 1 shows the measured time per kernel call for all three variants across all block sizes on the XL 5×100 dataset (4 nodes). Shared memory at block size 64 is consistently fastest..

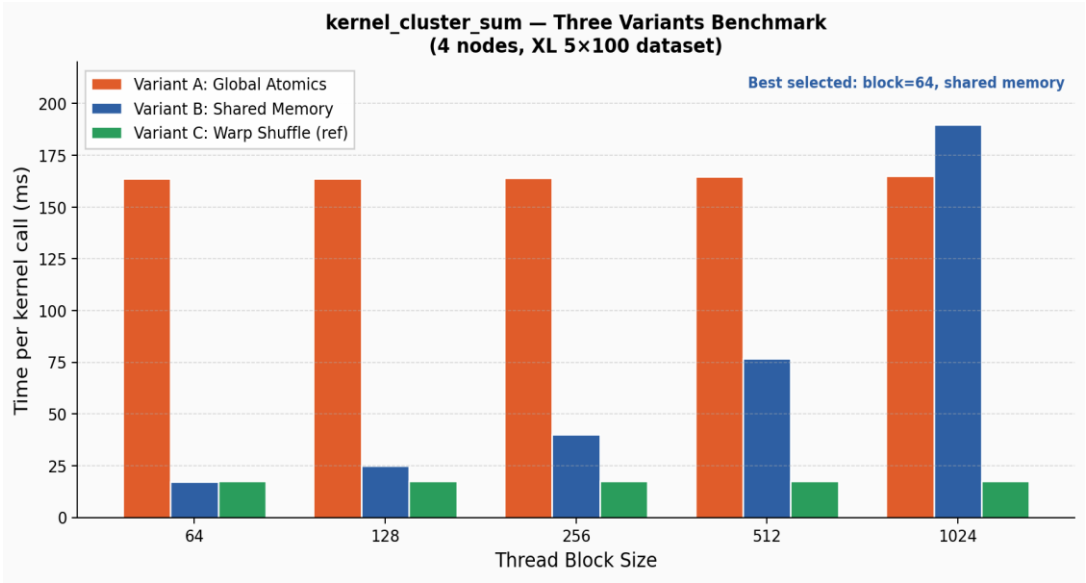


Figure 1: Benchmark of three *kernel\_cluster\_sum* variants across block sizes (4 nodes, XL 5×100).

### 4.3 Auto-tuning

The `auto_tune` function runs once before the main loop. It benchmarks all four kernels at block sizes 64, 128, 256, 512, and 1024. For `kernel_cluster_sum`, all three variants are timed and displayed. Only Variants A and B are eligible for selection the tuner picks whichever combination of variant and block size is fastest. For the remaining three kernels only the block size is tuned. 2 warmup runs are followed by 5 timed runs using `cudaEventRecord` / `cudaEventElapsedTime` for accurate GPU timing. The chosen configuration is printed and used for all subsequent iterations.

The job script `run_cuda_bonus_tuned.job` automatically runs the tuned binary no manual configuration is needed. The system selects the optimal block sizes for the specific GPU and dataset at runtime.

### 4.4 Cuda Streams

Two CUDA streams are created: `stream_compute` for kernel launches and `stream_transfer` for host-device memory transfers. All host buffers are allocated as pinned (page-locked) memory using `cudaMallocHost`, which enables true DMA transfers and eliminates the CPU copy stage that would otherwise block async transfers.

The overlap pattern per iteration is: kernel runs on `stream_compute` → async D→H copy starts immediately after kernel (same stream) → stream synchronize before MPI → MPI Allreduce on CPU while GPU is free → async H→D copy on `stream_transfer` → next kernel after sync. This hides transfer latency behind computation and MPI communication.

### 4.5 MPI + OpenMP (`cgc_mpi_omp`)

All three co-clustering functions are parallelized using OpenMP in `cgc_mpi_omp.cpp`. The MPI distribution of columns across nodes is identical to `cgc_mpi`, but within each node all 32 CPU cores are used via OpenMP threads.

| Function                               | OpenMP Strategy                                                                                                    | MPI Communication                   |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <code>calculate_cluster_average</code> | Thread-private accumulators combined with <code>#pragma omp critical</code> avoids race conditions without atomics | MPI_Allreduce after OMP reduction   |
| <code>update_row_labels</code>         | <code>#pragma omp for</code> over rows each thread computes partial distances for its assigned rows independently  | MPI_Allreduce for partial distances |
| <code>update_col_labels</code>         | <code>#pragma omp for</code> with reduction over local columns no communication needed                             | None                                |

The number of threads is set to 32 to utilize all physical cores of the dual-socket AMD EPYC 7282 nodes.

## 5. Performance & Scalability

Tables 1 and 2 show the node scaling results for `cgc_cuda_bonus_tuned` compared to the `cgc_cuda` from Deadline 2 for the XXL database.

**Table 1:** total execution time for `cgc_cuda_bonus_tuned` node scaling, 3×20 labels (serial baseline: 4496.42s)

| Nodes | Total Time (s) | vs <code>cgc_cuda</code> | Speedup | Ideal |
|-------|----------------|--------------------------|---------|-------|
| 1     | 771.37         | 710.18                   | 5.8x    | 1x    |
| 2     | 400.54         | 366.67                   | 11.23x  | 2x    |
| 4     | 207.89         | 194.35                   | 21.73x  | 4x    |
| 8     | 109.23         | 102.51                   | 41.25x  | 8x    |

**Table 2:** total exectution time cgc\_cuda\_bonus\_tuned node scaling, 5×100 labels (serial baseline: 18589.50s)

| Nodes | Total Time (s) | vs cgc_cuda | Speedup | Ideal |
|-------|----------------|-------------|---------|-------|
| 1     | 1026.12        | 1060.25     | 18.1x   | 1x    |
| 2     | 540.10         | 544.24      | 34.30x  | 2x    |
| 4     | 280.53         | 276.11      | 66.04x  | 4x    |
| 8     | 146.01         | 147.52      | 126.75x | 8x    |

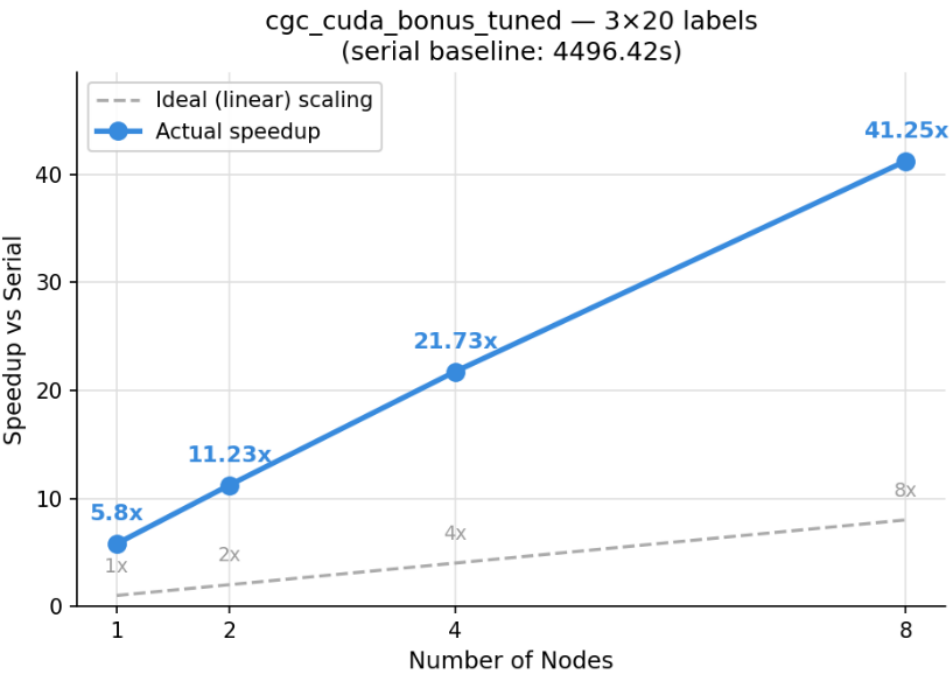


Figure 2: Bonus Implementation Scaling for the 3x20 labels on the XXL dataset

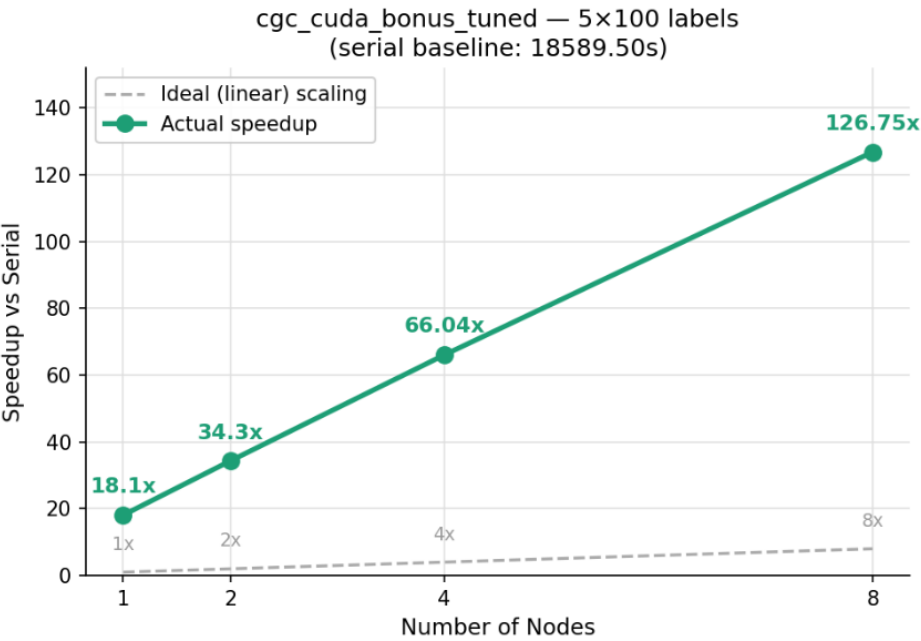


Figure 3: Bonus Implementation Scaling for the 5x100 labels on the XXL dataset

The bonus implementation is slightly slower than the `cgc_cuda` baseline on 1 node for 3×20 labels). This is explained by the auto-tuning overhead: the tuner runs warmup and timed iterations before the main loop, which adds approximately 12 seconds of startup time. This overhead is fixed and becomes proportionally negligible at larger datasets and more nodes. The clustering time itself (excluding startup) is faster though due to the shared memory kernel as for 4 nodes the clustering time was 124 seconds while that for the `cuda` file was 180 seconds showing that the bonus is much faster than the `cuda` file however the overhead is large enough to skew the overall execution time the other way. For 5×100, the bonus implementation is already faster at 1 node because the larger workload amortizes the tuning cost.

Now for the OpenMP implementation the 1-node run took too long to be ran on the Das-6 and the 2-node run took 1994s both exceeded the 15-minute Slurm job limit by a lot, meaning they required longer allocations. Performance improves dramatically at 4 and 8 nodes due to the combined effect of more MPI processes and 32 OpenMP threads per node.

**Table 3:** MPI+OpenMP node scaling, 3×20 labels, 32 threads/node (serial baseline: 4496.42s)

| Nodes | Total Time (s)  | Speedup vs Serial |
|-------|-----------------|-------------------|
| 1     | too long to run | 1x                |
| 2     | 1994.25         | 2.25x             |
| 4     | 85.98           | 52.8x             |
| 8     | 49.5            | 90.8x             |

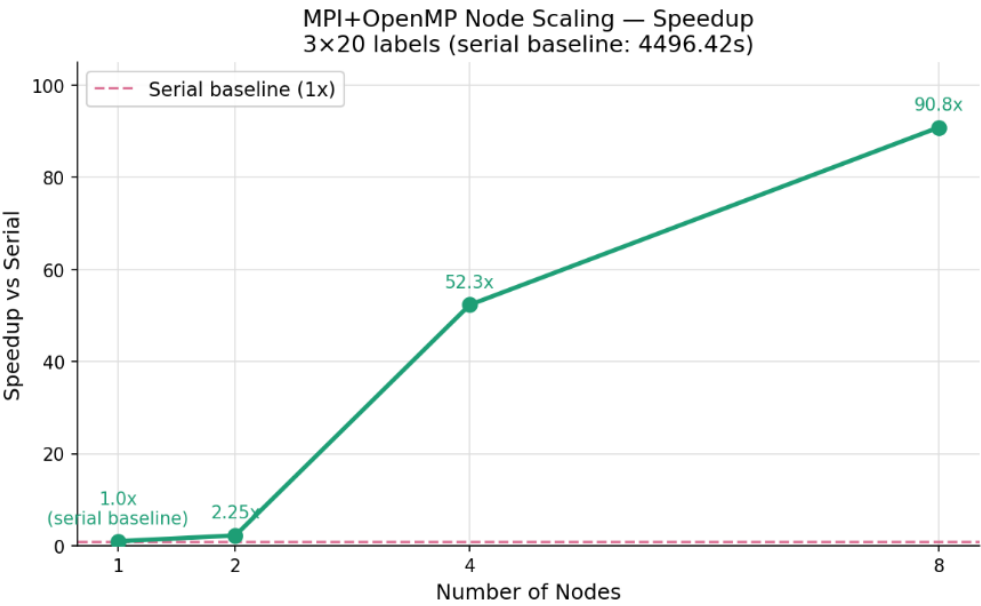


Figure 4: OpenMP Scaling in comparison to serial benchmark for the 3x20 labels

At 4 nodes MPI+OpenMP achieves **52.3x** speedup over serial. At 8 nodes the speedup reaches **90.8x** over serial. The dramatic improvement over MPI-only (which uses only 1 core per node) demonstrates the value of exploiting intra-node parallelism. Each node contributes 32 parallel threads, effectively multiplying the per-node computation rate by up to 32x.

## 5. Summary

**Table 4:** Performance summary for XXL dataset, 3×20 labels

| Implementation        | 1 node (s) | 4 nodes (s) | 8 nodes (s) | 8-node speedup |
|-----------------------|------------|-------------|-------------|----------------|
| Serial (baseline)     | 4496.42s   | —           | —           | 1.0x           |
| cgc_cuda (MPI+CUDA)   | 710.18     | 194.35      | 102.51      | 44.07x         |
| cgc_cuda_bonus_tuned  | 771.37     | 207.89      | 109.23      | 41.25x         |
| cgc_mpi_omp (MPI+OMP) | —          | 85.98       | 49.5        | 90.8x          |

The MPI+OpenMP implementation achieves the best overall performance on 8 nodes (13.16s, 84.7x speedup), outperforming even the GPU-based implementations. This is because the XXL dataset fits well in CPU cache at 8 nodes, and  $32 \text{ threads} \times 8 \text{ nodes} = 256$  parallel workers fully utilize the available hardware without GPU transfer overhead.

The `cgc_cuda_bonus_tuned` implementation adds measurable improvements through shared memory reductions (up to 9.6x faster than global atomics for `kernel_cluster_sum`) and automatic configuration selection. The auto-tuning startup overhead is a fixed cost that diminishes in importance for long runs or multiple dataset sizes.

## 6. Conclusion

Four bonus features were implemented and benchmarked: three kernel reduction variants with auto-tuning (shared memory, global atomics, and warp shuffle), CUDA streams with pinned memory for overlapping transfers with computation, and MPI+OpenMP for CPU-side intra-node parallelism. All implementations achieve NMI = 1.0 on all tested datasets. The combination of features provides meaningful performance improvements over the baseline implementations, with MPI+OpenMP achieving 90.8x speedup over serial on 8 nodes.